Swarm Intelligence and Target Search

Alessandro Rossi[1] **and Lorenzo Franco**[2]

[1]Intelligent Distributed Systems, master's degree in mechatronics engineering, University of Trento

Abstract – A "treasure hunt" can be a fun and engaging opportunity to test the capabilities of a swarm of robots in a simulated environment. The multiagent system proposed in this project is designed to search for a target in a 2D environment using a combination of swarm intelligence (? ?), Voronoi cells (? ? ?), Lévy flight (? ?) and sensor fusion techniques. The main objective of the project is to evaluate the performance of the proposed multiagent system in terms of search efficiency, convergence time, and robustness to noise and uncertainties in the environment. The simulation will be implemented in Python, and the results will be analyzed to draw conclusions about the effectiveness of the proposed approach in solving the treasure hunt problem.

Introduction

This project presents a control framework for a swarm of mobile robots that finds, estimates and encircles a target. The system combines a lidar, for obstacle detection, an Extended Kalman Filter (EKF) for state estimation and linear consensus for coordination. The treasure is supposed to be a point object emitting a weak radio signal, the detection is performed by a simple thresholding of the received signal strength. The robots are able to coordinate their movements using a linear consensus algorithm, allowing convergence to the estimated position of the target while avoiding obstacles. An iteration of the consensus algorithm enhances the precision of the target estimate. The performance of the proposed control framework is evaluated through simulations in a 2D environment, where the robots are tasked with finding and aligning around a target while navigating through a cluttered environment. The results show that the proposed approach is effective in enabling the swarm of robots to find and align around the target, even in the presence of noise and uncertainties in the environment.

The Task

The objective of this project is the localization of a target, modeled as a point object, within a two-dimensional environment. The problem is framed in the context of distributed cooperative control for a team of autonomous mobile robots.

The proposed framework integrates three main components: a Voronoi–Lloyd partitioning scheme, an Extended Kalman Filter (EKF), and a linear consensus algorithm. The EKF is employed for sensor fusion, combining ead-reckoning data with camera-based position measurements. The robotic swarm is required to explore the environment, avoid obstacles, detect the target, and finally converge and align around it.

Structure and Workflow. The simulation takes place in a 2D environment populated with obstacles, representing a cluttered indoor scenario. A finite state machine (FSM) governs the behavior of each robot. The operational states are defined as follows:

- Searching: Robots explore the environment using a Lévy flight strategy, while actively avoiding collisions with obstacles and other agents.
- Inspecting: Upon detecting an obstacle, a robot performs local mapping. It also verifies whether the detected object is the target by approaching it and checking for the presence of a radio signal.
- Target Found: Once the target is identified, all agents converge toward its estimated position. The estimate is progressively refined through iterations of the consensus algorithm.

State estimation is achieved via sensor fusion using the EKF, which combines velocity-based dead-reckoning with measurements from a fixed camera located in the top-right corner of the environment (see Figure **??**).
Voronoi cells are computed and used within the control logic of the FSM to improve spatial coordination among robots.
Additionally, agents communicate with one another by broadcasting messages that include: the position estimate provided by the EKF, locally mapped points contributing to a global map of the environment, and, once detected, the estimated position of the target.
The number of agents, the obstacles shape and position, the room size and the target position are user defined.
Each aspect described above will be discussed in the following sections of the report.

Model and methods

A. Motion Model. The agents operate in a planar environment and are therefore modeled in two dimensions. The state of each robot is defined as:

$$\mathbf{s} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix} \tag{1}$$

where (x, y) represents the position of the robot in the plane, and θ denotes its orientation with respect to a global reference frame.

To describe the motion of the robots, the unicycle kinematic model is adopted. The corresponding nonlinear dynamics are given by:

$$\dot{\mathbf{s}} = \begin{bmatrix} \dot{x} = v\cos\theta \\ \dot{y} = v\sin\theta \\ \dot{\theta} = \omega \end{bmatrix} \tag{2}$$

where v is the linear velocity and ω is the angular velocity, which together form the control input vector:

$$\mathbf{u} = \begin{bmatrix} v \\ \omega \end{bmatrix} \tag{3}$$

The unicycle model represents a simplified yet realistic approximation of the robot dynamics. In particular, it captures the nonholonomic constraint that prevents lateral motion, meaning the robot cannot move directly sideways. Despite its simplicity, this model is widely used in mobile robotics, as it effectively captures the behavior of differential-drive robots and similar platforms, while remaining computationally efficient for control and estimation purposes. For these reasons, it was selected for this work.

To improve the realism of the model, disturbances are introduced in the control inputs (Eq.**??**). In practice, uncertainties in differential-drive robots mainly arise from actuation errors, such as wheel slippage, surface irregularities, and imperfect motor response.

The discrete-time version of Eq. **??** are therefore modeled as:

$$\mathbf{s}_{k+1} = f(\mathbf{s}_k, \mathbf{u}_k, \boldsymbol{\nu}_k) \tag{4}$$

where the process noise $\boldsymbol{\nu}_k$ is assumed Gaussian:

$$\boldsymbol{\nu}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \tag{5}$$

The covariance matrix is chosen diagonal:

$$\mathrm{Q} = \begin{bmatrix} \sigma_v^2 & 0 \\ 0 & \sigma_\omega^2 \end{bmatrix} \tag{6}$$

Thus, the relation (Eq. **??**) can be written as:

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{bmatrix} = \begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix} + \begin{bmatrix} \cos\theta_k & 0 \\ \sin\theta_k & 0 \\ 0 & 1 \end{bmatrix} \Delta t \left(\begin{bmatrix} v_k \\ \omega_k \end{bmatrix} + \boldsymbol{\nu}_k \right) \tag{7}$$

B. Measurement and Sensing Model. The robots are assumed to be equipped with the following sensing systems:

- Camera-based position measurements: used in the update step of the EKF. A fixed camera observes the robots (e.g., through ArUco markers) and provides an estimate of their full pose in the global reference frame.

  The measurement model is therefore defined as:

  $$\mathbf{z}_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix} + \epsilon_k = H \cdot \mathbf{s} + \epsilon_k \tag{8}$$

  where ϵ_k represents the measurement noise, assumed to be Gaussian:

  $$\epsilon_k \sim \mathcal{N}(\mathbf{0}, R) \tag{9}$$

  The covariance matrix R models the uncertainty of the vision system and is modeled as the diagonal matrix:

  $$R = \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_\theta^2 \end{bmatrix} \tag{10}$$

- LiDAR sensor: used to perceive the environment (obstacles and target) and to construct the Voronoi cells by measuring the distance between the robot and surrounding objects. The distance measurement is affected by Gaussian noise and can be modeled as:

  $$z_d = d + n_d \tag{11}$$

  where d is the true distance and n_d is the measurement noise:

  $$n_d \sim \mathcal{N}(0, \sigma_d^2) \tag{12}$$

  This measurement is also used, together with the EKF estimate, to build the environment map as will be described in the following.
- Radio receiver: used to detect the signal emitted by the target. This sensor provides a binary measurement indicating the presence or absence of the signal within a certain detection range.

C. Communication System. The agents exchange data packets, as previously described, to share their state estimates, mapped points, and target information. To simulate a realistic communication scenario, the network is modeled similarly to (**?**).

Specifically, suppose the clock of node i ticks at time step t. Node i then broadcasts its current payload to all the others. Each agent transmits its data using a Time Division Multiple Access (TDMA) scheme, dividing the transmission channel into distinct time slots so that agents send sequentially in their assigned slot.

We also assume that all data in the payload is available at the transmission instant. While these assumptions simplify the simulation, we acknowledge they are not fully realistic; especially the TDMA protocol, as it requires a level of global

clock coordination that may conflict with the pure decentralization intended for such systems.
The network topology is modeled as a complete graph; therefore, each robot can always communicate with all the other agents in the swarm, as illustrated in Fig. **??**.

Implementation

D. Extended Kalman Filter. Each robot estimates its own pose using an Extended Kalman Filter (EKF). This approach enables handling the nonlinear system dynamics while fusing information from multiple sensors. In particular, the EKF provides a framework to combine the internal dead-reckoning estimation with global camera-based measurements, explicitly accounting for both process and measurement uncertainties.
In our implementation, the prediction step of the EKF relies on a velocity motion model based on unicycle kinematics. The state is propagated forward using the commanded linear and angular velocities (v and ω) directly as control inputs. Since physical actuators are imperfect and subject to slippage or mechanical tolerances, we explicitly model this actuation uncertainty by injecting Gaussian process noise into the velocity commands. This dead-reckoning approach is effective for short-term estimation; however, it inevitably accumulates drift over time, which is subsequently corrected using the camera observations during the update step.
The algorithm runs in two main phases at each iteration: a prediction and an update step.

1) Prediction Step. In this phase, the algorithm exploits the knowledge of the current state estimate and the applied control input to forecast the state at the next time step using the system's non-linear dynamics (Eq.**??**). We have:

$$\hat{s}_{k+1}^{-} = f(\hat{s}_k, u_k), \tag{13}$$

where f is the model dynamics, $\hat{s}_k$ is the current state estimate, and u_k is the applied control input.
Moreover, the covariance matrix is also propagated. Since the system model is non-linear, linearization around the current estimate is required:

$$P_{k+1}^{-} = A_k P_k A_k^T + G_k Q G_k^T, \tag{14}$$

with A_k and G_k denoting the Jacobians of the dynamics with respect to the state and the process noise at the current time step k, respectively:

$$A_k = \left.\frac{\partial f(s,u,\nu)}{\partial s}\right|_{\substack{s=\hat{s}_k \\ u=u_k \\ \nu=0}}, \quad G_k = \left.\frac{\partial f(s,u,\nu)}{\partial \nu}\right|_{\substack{s=\hat{s}_k \\ u=u_k \\ \nu=0}} \tag{15}$$

Computing explicitly the Jacobians of Eq. **??** on the unicycle model, we obtain:

$$A_k = \begin{bmatrix} 1 & 0 & -v_k \sin(\hat{\theta}_k)\Delta t \\ 0 & 1 & v_k \cos(\hat{\theta}_k)\Delta t \\ 0 & 0 & 1 \end{bmatrix}, \quad G_k = \begin{bmatrix} \Delta t \cos(\hat{\theta}_k) & 0 \\ \Delta t \sin(\hat{\theta}_k) & 0 \\ 0 & \Delta t \end{bmatrix} \tag{16}$$

2) Update Step. Following the prediction phase, the EKF integrates the sensor measurements available at the current time step to refine the state estimate. In our framework, the sensing modality consists of a fixed global camera that provides direct observations of the robot's full pose (both 2D position and orientation). Since the sensor directly measures all the components of the state vector $\hat{s}_{k+1} = [x_{k+1}, y_{k+1}, \theta_{k+1}]^T$, the measurement model is linear. The measurement matrix H is therefore the 3×3 identity matrix:

$$H = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{17}$$

Whenever a camera measurement $z_{k+1} \in \mathbb{R}^3$ becomes available at instant $k+1$, the EKF first computes the innovation:

$$y_{k+1} = z_{k+1} - H\hat{s}_{k+1}^{-}, \tag{18}$$

which quantifies the discrepancy between the received measurement z_{k+1} and its predicted value. The covariance of the innovation is computed as:

$$S_{k+1} = H P_{k+1}^{-} H^T + R_{k+1}, \tag{19}$$

where R_{k+1} denotes the 3×3 measurement noise covariance matrix associated with the camera (Eq. **??**). The Kalman gain is then evaluated as:

$$W_{k+1} = P_{k+1}^{-} H^T S_{k+1}^{-1}. \tag{20}$$

The corrected state estimate is subsequently obtained through:

$$\hat{s}_{k+1} = \hat{s}_{k+1}^{-} + W_{k+1} y_{k+1}, \tag{21}$$

followed by the covariance matrix update:

$$P_{k+1} = (I - W_{k+1} H) P_{k+1}^{-}. \tag{22}$$

This localization framework proves to be highly robust to temporary sensory loss. In scenarios where the robot is occluded by an obstacle and the global camera cannot provide a valid measurement, the system does not fail; instead, the EKF smoothly falls back on its prediction phase. State estimation is temporarily maintained via velocity-based dead-reckoning, and any accumulated drift is immediately corrected as soon as the robot re-enters the camera's field of view.

E. Voronoi-Lloyd Based Partitioning. The prerequisite of this framework is to provide a distributed control logic that works reliably and effectively in a crowded environment, enabling the swarm to explore the area, avoid obstacles, and converge around a target. For this purpose a Voronoi space partitioning was adopted, along with Lloyd-based navigation (**?**) to converge to the target.
Let $Q \in \mathbb{R}^2$ be the mission space, q a generic point in the Q space, and p_r the robot position. Given the set of agents and their positions $\mathcal{P} = \{p_1, \ldots, p_n\}$, the i-th Voronoi cell can be computed as follows:

$$V_i = \{q \in Q \mid \|q - p_i\| \leq \|q - p_j\|, \forall j \neq i\} \tag{23}$$

Equation **??** assigns each robot located at p_i a portion of the map: its Voronoi cell V_i. This region contains all points that are closer to robot i than to all other robots, intrinsically granting inter-robot collision avoidance.

2) Reactive Control for Static Obstacles. The Voronoi partitioning alone does not prevent collisions with static obstacles. To enforce this, the cell V_i is constrained by the robot's sensing radius r_s and line-of-sight visibility.

We define the i-th visibility set S_i using a ray-casting approach, where the line-of-sight is interrupted if it intersects an obstacle:

$$S_i = \{q \in V_i \mid p_i - \gamma(p_i - q) \notin \mathcal{O}\}, \ \ \forall \gamma \in [0,1] \qquad \textbf{(24)}$$

where $\mathcal{O} \subset Q$ is the obstacle set, i.e., the set of points in the map corresponding to static obstacles.

At this point, the resulting visibility set S_i is intersected with the Voronoi cell, reducing it to the intermediate set $\bar{W}_i = V_i \cap S_i$. The reduced cell $\bar{W}_i$ is then bounded by the physical sensing radius of the robot r_s, yielding the final safe navigation cell W_i:

$$W_i = \{q \in \bar{W}_i \mid \|q - p_r\| \leq r_s\} \qquad \textbf{(25)}$$

In practice, the visibility set and sensing limit r_s are computed simultaneously using a ray-casting algorithm similar to what presented in (**?**) implemented via the Python `Shapely` library.

Radial rays, spaced by the angular resolution set by the LiDAR, are cast from the robot up to a distance r_s and geometrically truncated at the exact point of intersection with any obstacle.Connecting the endpoints of these rays creates a strictly bounded visibility polygon, S_i. The final safe navigation cell W_i is then efficiently obtained through a geometric boolean intersection:

$$W_{r_s} = V_i \cap S_i \qquad \textbf{(26)}$$

To accurately simulate the characteristics of a real LiDAR sensor, noise is injected into the computed ray lengths, as described in Eq. **??**.

Another aspect to consider is that each agent requires the positions of its peers to compute its local Voronoi cell. To achieve this, every robot periodically shares its estimated position with the rest of the swarm via the previously described broadcast mechanism. This approach introduces an important layer of realism to the simulation, as the robots do not possess instantaneous, continuous knowledge of each other's true states. Instead, the spatial partitioning relies entirely on the last known positions received from the network, accurately reflecting the discrete nature of real-world communication systems.

The outcome of this decentralized partitioning is illustrated in Fig. **??**. As can be observed, the boundaries of the resulting Voronoi cells are not perfectly smooth, which is a direct consequence of the simulated LiDAR measurement noise. Furthermore, the spatial separation between the robots is slightly asymmetrical; this reflects the communication delays, as each agent computes its cell based on the periodically broadcasted, and therefore slightly outdated, positions of its peers.

F. Control FSM. To govern the behavior of the agents according to their current objectives, a Finite State Machine (FSM) was developed. Before detailing the specific states and transitions of the FSM, we first mathematically formulate the underlying control laws that dictate the robots' kinematics ($[v, \omega]^T$) in various scenarios: exploration, obstacle avoidance, boundary mapping, and target formation.

1) Lévy Flight Control. During the search phase, pure random walks are highly inefficient. Instead, the robots employ a Lévy flight exploration strategy. This method mimics the optimal foraging behaviors naturally observed in the animal kingdom, such as those of marine predators searching for prey (**?**). Taking inspiration from the framework presented in (**?**), a new target heading θ_{target} is chosen uniformly at random, while the duration for which the robot maintains this heading is periodically sampled from a stable Lévy distribution, as illustrated in Fig. **??**.

To ensure practical applicability, minimum and maximum temporal bounds are enforced on the sampled values, which are also reported in the figure. These boundaries and the distribution parameters can be easily fine-tuned to better suit the scale and specific requirements of the mission at hand. This time-based sampling generates a characteristic movement pattern consisting of many short, localized searches interspersed with occasional long-distance relocations, allowing the swarm to cover large areas efficiently. So during a time interval a vector $\vec{v}_{levy}$ is generated as:

$$\vec{v}_{levy} = \begin{bmatrix} \cos\theta_{target} \\ \sin\theta_{target} \end{bmatrix} \qquad \textbf{(27)}$$

2) Obstacle Avoidance Control. To prevent collisions during exploration, a repulsive approach is adopted. We implement a quadratic bounded repulsive force.

Let $d = \|p - p_{vor}\|$ be the Euclidean distance from the robot's position p to the closest point p_{vor} on its Voronoi. If this distance falls below a critical safety threshold d_{min}, a repulsive vector $\vec{v}_{rep}$ is generated:

$$\vec{v}_{rep} = \begin{cases} k_{rep}(d_{min} - d)^2 \frac{p - p_{obs}}{d} & \text{if } d < d_{min} \\ \mathbf{0} & \text{if } d \geq d_{min} \end{cases} \qquad \textbf{(28)}$$

where k_{rep} is the repulsive gain. The fraction $\frac{p - p_{obs}}{d}$ computes the normalized directional vector pointing away from the obstacle.

3) Wall Following Control. When a robot encounters an obstacle, it enters an inspection phase to map its perimeter. To do so, this controller makes the robot slide along the obstacle's boundary. To achieve a smooth sliding behavior while maintaining a safe distance, the control vector is computed as the superposition of a tangential component along the boundary and a normal distance correction.

Let $\vec{n}$ be the normalized vector pointing from the robot's position p to the closest point on its Voronoi cell boundary (which represents the closest obstacle), p_{obs}, defined as $\vec{n} =$

$\frac{p_{obs}-p}{\|p_{obs}-p\|}$. A tangent vector $\vec{t}$ is derived by rotating $\vec{n}$ by 90 degrees: $\vec{t} = [-m_{dir}n_y, m_{dir}n_x]^T$, where $m_{dir} \in \{1, -1\}$ dictates the sliding direction (counter-clockwise or clockwise). This direction is chosen randomly whenever a new obstacle is encountered.

To ensure the robot does not drift away or crash, a proportional controller corrects the distance error $e = \|p_{obs} - p\| - d_{target}$, where d_{target} is the desired safe distance from the wall. The desired movement vector is thus formulated as:

$$\vec{v}_{des} = \begin{bmatrix} v_{des,x} \\ v_{des,y} \end{bmatrix} = \vec{t} + K_p \cdot e \cdot \vec{n} \tag{29}$$

where K_p is the proportional gain.

The robot aligns its motion with this vector's angle, $\theta_{target} = \arctan 2(v_{des,y}, v_{des,x})$, utilizing the following control law:

$$u_{wall} = \begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} V_{lin} \cos^3(\theta_{target} - \theta)) \\ K_\omega(\theta_{target} - \theta) \end{bmatrix} \tag{30}$$

where V_{lin} is the maximum linear velocity, K_ω is the angular velocity gain, and θ is the current heading of the robot.

Notably, the linear velocity is modulated by the cubed cosine of the heading error ($\cos^3$). This ensures a highly responsive behavior: if the angle error is large, the forward speed drops dramatically, allowing the robot to perform tight corrective turns before resuming its sliding motion along the wall.

4) Formation Control and Lloyd's Algorithm. Once the target is located, the swarm transitions from exploration to a coordinated formation phase. To achieve a stable and distributed configuration around the target, we employ a Voronoi-based coverage strategy known as Lloyd's Algorithm (**? ?**), modified to incorporate environmental constraints and a specific spatial density.

The fundamental objective is to distribute the agents according to a desired spatial density, defined by a Probability Density Function $\phi(q)$. Our framework follows the approach presented in (**?**), where $\phi(q)$ is designed as a ring-shaped PDF (Fig.**??**) centered at the estimated target position p_{target}. Specifically, this distribution is defined as:

$$\phi(q) = \exp\left(-\frac{(\|q - p_{target}\| - R_{ring})^2}{2\sigma^2}\right) \tag{31}$$

where q is a generic point in the mission space Q and R_{ring} is the desired radius of the circular formation. If $R_{ring} = 0$, the distribution collapses to a standard Gaussian centered on the target. To improve stability and prevent oscillations when agents are far from the goal, an adaptive standard deviation σ is used:

$$\sigma = \sigma_0 \left(1 + \frac{\|p - p_{target}\|}{R_{ad}}\right) \tag{32}$$

where σ_0 is the base spread and R_{ad} is a scaling distance.

For each agent i, we compute the "mass" M_{V_i} and the weighted centroid C_{V_i} of its Voronoi cell V_i as:

$$M_{V_i} = \int_{V_i} \phi(q)\, dq, \quad C_{V_i} = \frac{1}{M_{V_i}} \int_{V_i} q\phi(q)\, dq \tag{33}$$

The centroid C_{V_i} represents the local optimal position. Thus, we can define the Lloyd-based target vector as follows:

$$\vec{v}_{target} = \begin{bmatrix} v_{target,x} \\ v_{target,y} \end{bmatrix} = (C_{V_i} - p) + k_{rep} \cdot \vec{v}_{rep} \tag{34}$$

where $\vec{v}_{rep}$ is the repulsive vector from nearby obstacles. While the standard Lloyd control is inherently robust, the addition of the repulsive term ensures an extra layer of safety, allowing the robots to navigate more reliably around obstacles during the formation process.

Finally, the robot's velocities are generated using the following control law:

$$u_{lloyd} = \begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} V_{lin} \cos^3(\theta_{target} - \theta) \\ K_\omega(\theta_{target} - \theta) \end{bmatrix} \tag{35}$$

where $\theta_{target} = \arctan 2(v_{target,y}, v_{target,x})$. This formulation allows the swarm to converge smoothly into a ring formation while maintaining safety distances from boundaries.

With the underlying control laws mathematically defined, we can now detail the Finite State Machine (FSM) that dictates the high-level behavior of each agent. The FSM is designed to seamlessly manage the transitions between exploration, obstacle mapping, and target formation. It consists of three primary states:

1) Searching. This is the default exploration state. In this phase, the agent navigates the environment using the Lévy flight exploration strategy. To ensure safe navigation, the desired Lévy direction is continuously superimposed with the obstacle avoidance control vector. The resulting direction vector is thus defined as the combination of Eq.**??** and Eq.**??**:

$$\vec{v}_{search} = \begin{bmatrix} v_{search,x} \\ v_{search,y} \end{bmatrix} = \vec{v}_{levy} + \vec{v}_{rep} \tag{36}$$

From this combined vector, the new target heading is computed using the four-quadrant inverse tangent:

$$\theta_{target} = \text{arctan2}(v_{search,y}, v_{search,x}) \tag{37}$$

To prevent the robot from moving forward while correcting large angular deviations—which could lead to unsafe behaviors near obstacles—a piecewise control law is applied to the linear velocity, while the angular velocity remains proportional to the error:

$$u_{search} = \begin{bmatrix} v \\ \omega \end{bmatrix} = \begin{bmatrix} \begin{cases} 0 & \text{if } |e_\theta| > \frac{\pi}{4} \\ V_{lin}\cos(e_\theta) & \text{otherwise} \end{cases} \\ K_\omega\, e_\theta \end{bmatrix} \tag{38}$$

Where $e_\theta = (\theta_{target} - \theta)$ is the angular error. This control strategy allows the robot to rotate in place when a sharp turn is required (e.g., when suddenly repelled by a boundary), and perform smooth, modulated (by the cosine) curves otherwise. The robot continues this exploratory behavior until its simulated LiDAR detects a physical obstacle within a predefined distance threshold. If the detected obstacle points have not been previously mapped by the swarm, the FSM transitions the robot to the `INSPECTING` phase.

2) Inspecting. Upon encountering an unmapped obstacle, the robot temporarily suspends random exploration and enters the `INSPECTING` state. Here, the FSM switches to the wall-following control strategy u_{wall} (Eq.**??**). The agent slides along the contour of the obstacle at a safe distance d_{target}, collecting new boundary points and continuously updating the swarm's shared map.
The robot reverts to the `SEARCHING` state under three specific conditions: if it detects a loop closure (indicating it has fully circumnavigated the obstacle or merged with an already mapped area), if it reaches a critical proximity to another robot (triggering a collision avoidance maneuver), or if it simply loses track of the wall..

3) Treasure Found. This terminal phase is activated when the robot either enters the physical sensing range of the target object or receives the target's estimated coordinates through the swarm's consensus network. Once activated, both exploration and mapping halt entirely, and the FSM delegates the kinematic commands to the Voronoi-based Formation Control, u_{lloyd} (Eq.**??**). In this state, the agent converges toward the target and begins to arrange itself into the ring formation. While maintaining this formation, the robot continuously computes a weighted average of the target's position to build a shared consensus estimate, maneuvering to distribute itself evenly along the desired circular formation defined by the ring-shaped probability density function.

G. Collaborative Mapping. The mapping process begins with measurements acquisition through the LiDAR sensor. For each agent, a raycasting algorithm is executed in the same way as described in the Voronoi discussion (**?**). These intersections yield a set of local measurements z_d (Eq.**??**) and their relative angle α, representing the range and bearing of the detected points relative to the robot's sensor frame.
It is worth highlighting the realistic nature of the mapping mechanism employed to process these raw readings. In real-world applications, a robot perceives the environment relative to its true physical location, but it must register these sensor readings in a global coordinate frame relying solely on its estimated state. To faithfully replicate this paradigm, the simulated LiDAR captures measurements originating from the robot's ground-truth state, defined as $\mathbf{s} = [x, y, \theta]^T$.
However, because $\mathbf{s}$ is unknown to the robot, the mapping algorithm intentionally integrates the physical measurements (z_d, α) exclusively with the robot's current state estimate provided by the Extended Kalman Filter (EKF), denoted as $\hat{s} = [\hat{x}, \hat{y}, \hat{\theta}]^T$. To compute the absolute global coordinates of the newly discovered boundary points, p_{map}, a rigid body transformation is applied using the estimated state:

$$p_{map} = \begin{bmatrix} \hat{x} \\ \hat{y} \end{bmatrix} + \begin{bmatrix} \cos(\hat{\theta}) & -\sin(\hat{\theta}) \\ \sin(\hat{\theta}) & \cos(\hat{\theta}) \end{bmatrix} \begin{bmatrix} z_d \cos(\alpha) \\ z_d \sin(\alpha) \end{bmatrix} = \begin{bmatrix} \hat{x} + z_d \cos(\hat{\theta} + \alpha) \\ \hat{y} + z_d \sin(\hat{\theta} + \alpha) \end{bmatrix} \tag{39}$$

As a result, the computed points p_{map} are affected by the estimation error $e = x_{true} - \hat{x}_{ekf}$. The constructed shared map inherently reflects the localization uncertainties of the swarm, accurately modeling the intrinsic challenges of real-world robotic mapping where odometry drift directly distorts the generated map.
Furthermore, the shared map is distributed across the swarm as a component of the broadcasted communication payload. Consequently, the global map is not instantaneously synchronized across the memory of all agents. Instead, newly mapped boundary points propagate iteratively through the network as the robots exchange data. This delayed, decentralized synchronization further enhances the realism of the simulation, accurately reflecting the communication constraints and data propagation dynamics inherent to physical multi-robot systems.

Conclusions

Bibliography

. Tuncer Aysal, Mehmet Yildiz, Anand Sarwate, and Anna Scaglione. Broadcast gossip algorithms for consensus. *Signal Processing, IEEE Transactions on*, 57:2748 – 2761, 08 2009. doi: 10.1109/TSP.2009.2016247.
. Stephen Boyd and Sanjay Lall. A scheme for robust distributed sensor fusion based on average consensus. *2005 4th International Symposium on Information Processing in Sensor Networks, IPSN 2005*, 2005:63–70, 04 2005. doi: 10.1109/IPSN.2005.1440896.
. Manuel Boldrer, Luigi Palopoli, and Daniele Fontanelli. Lloyd-based approach for robots navigation in human-shared environments. pages 6982–6989, 10 2020. doi: 10.1109/IROS45743.2020.9341272.
. Colin Sauzé and Mark Neal. A raycast approach to collision avoidance in sailing robots. 06 2010.
. Donny Sutantyo, Serge Kernbach, P. Levi, and Valentin Nepomnyashchikh. Multi-robot searching algorithm using lévy flight and artificial potential field. *Safety Security and Rescue Robotics (SSRR)*, pages 1 – 6, 08 2010.
. David Sims, Emily Southall, Nicolas Humphries, Graeme Hays, Corey Bradshaw, Jonathan Pitchford, Alex James, M. Ahmed, Andrew Brierley, Mark Hindell, David Morritt, Michael Musyl, David Righton, Emily Shepard, Victoria Wearmouth, Rory Wilson, Matthew Witt, and J. Metcalfe. Scaling laws of marine predator search behaviour. *Nature*, 451:1098–102, 03 2008. doi: 10.1038/nature06518.

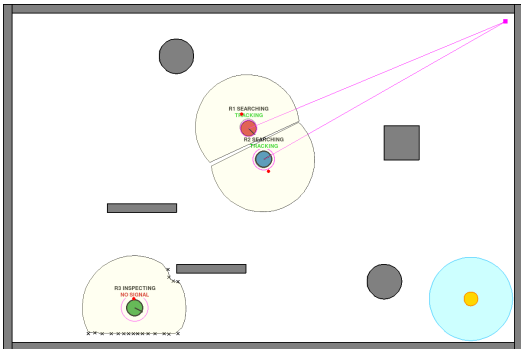


Fig. 1. Simulation environment

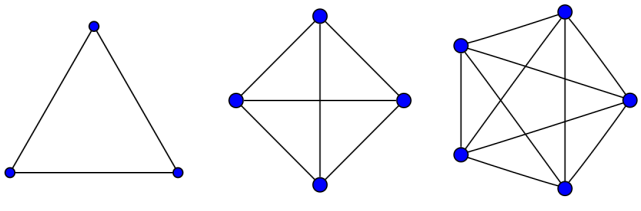


Fig. 2. complete graphs.

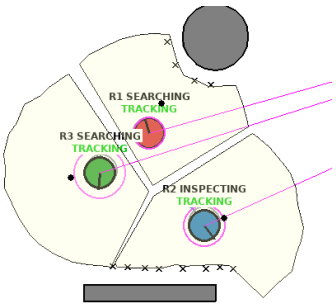


Fig. 3. Voronoi partition.

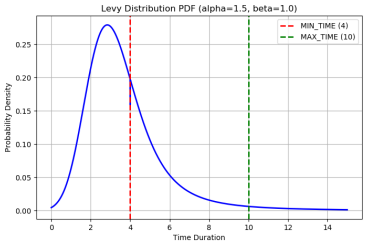


Fig. 4. Lévy distribution used.

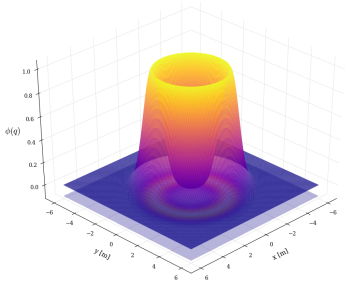


Fig. 5. Ring shaped PDF